LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 25

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 24, halaman View pertama menampilkan data mahasiswa dari API lewat Fetch API dan dapat dibuka siapa saja. Acara 25 menambahkan dua hal yang belum ada: pengguna harus *login* sebelum masuk ke halaman pengelolaan, dan hak akses dibedakan menurut peran. Kolom *role* pada tabel *users* yang sudah ada sejak Acara 19 kini dipakai oleh class Auth lewat method *requireAdmin()*, sehingga hanya admin yang boleh menambah, mengubah, dan menghapus data mahasiswa.

2. Tujuan

1. Menambahkan method requireAdmin() pada class Auth dan memakainya pada store(), update(), dan destroy() di MahasiswaController.
2. Membuat View login.html yang mengirim kredensial ke POST /login dan menyimpan token serta role pada localStorage.
3. Melindungi dashboard.html agar pengguna tanpa token diarahkan ke halaman login.
4. Menampilkan ringkasan jumlah mahasiswa pada dashboard.
5. Menguji alur login dan pembatasan akses berdasarkan role (RBAC).
6. Menambahkan tombol Logout (tugas mandiri 1).
7. Menyembunyikan tombol Tambah/Ubah/Hapus bagi role user (tugas mandiri 2).

3. Dasar Teori

Authentication adalah proses memverifikasi identitas pengguna melalui username dan password, sedangkan *authorization* menentukan hak akses pengguna setelah identitasnya terverifikasi. Sebuah sistem dapat berhasil mengautentikasi pengguna namun tetap menolak suatu aksi karena pengguna tidak memiliki otorisasi yang sesuai.
Role-Based Access Control (RBAC) membatasi hak akses berdasarkan peran pengguna. Pada proyek ini ada dua peran: *admin* yang dapat mengelola data (Create, Update, Delete) dan *user* yang hanya dapat melihat data. Dua kegagalan akses dibedakan dengan kode status berbeda (Tabel 1): 401 berarti identitas tidak dapat dipastikan, 403 berarti identitas dikenal tetapi tidak berhak.

Tabel 1. Perbedaan kode status 401 dan 403 pada API

Kode	**Penyebab**	**Contoh pesan**
401 Unauthorized	Token tidak ada, rusak, atau kedaluwarsa	Token tidak ditemukan; Token tidak valid atau kedaluwarsa
403 Forbidden	Token sah, tetapi role bukan admin	Akses ditolak, hanya admin yang diizinkan

Token JWT pada API ini berisi *id*, *role*, dan waktu kedaluwarsa (*exp*) lima menit. Browser menyimpannya pada *localStorage* agar tersedia di antara halaman. Isinya dapat dibaca skrip apa pun pada halaman yang sama, sehingga pencegahan *cross-site scripting* (XSS) menjadi penting.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 25 mengubah dua berkas backend dan menambahkan tujuh berkas View. Dokumentasi, koleksi Postman, dan skrip SQL untuk akun admin ikut diperbarui.

Tabel 2. Daftar perubahan file

File	Status	Keterangan
app/core/Auth.php	Diubah	Method requireAdmin()
app/controllers/MahasiswaController.php	Diubah	store(), update(), destroy() memakai requireAdmin()
public/views/common.js	Baru	BASE_URL, pengelola sesi, pemanggil API
public/views/login.html, login.js	Baru	Form login dan pengiriman ke POST /login
public/views/dashboard.html, dashboard.js	Baru	Ringkasan, tabel, Tambah/Ubah/Hapus, Logout
public/views/style.css	Baru	Gaya bersama login dan dashboard
public/views/logo.png	Baru	Logo pada halaman login
public/views/index.html, app.js	Diubah	Tautan Login admin dan BASE_URL ke folder acara25
db_seed_acara25.sql	Baru	Menjadikan akun admin
postman/ACARA25_TestCase...json	Baru	25 test case Acara 23 ditambah TC-26 sampai TC-29
docs/API_DOCUMENTATION.md, openapi.yaml, TEST_CASE.md	Diubah	Respons 403 dan hasil uji RBAC

b. Langkah 1 - Menambahkan Method requireAdmin() pada Class Auth

requireAdmin() memanggil requireAuth() lebih dulu, jadi token yang tidak ada atau tidak sah tetap dijawab 401. Hanya token yang sah dan berrole admin yang diteruskan; selain itu server menjawab 403. Pembacaan role memakai *??* agar token tanpa klaim role juga ditolak, bukan menimbulkan peringatan PHP.

```
// ditambahkan pada app/core/Auth.php
public static function requireAdmin(): array {
    $payload = self::requireAuth();
    if (($payload['role'] ?? '') !== 'admin') {
        Response::json(403, false, 'Akses ditolak, hanya admin yang diizinkan');
    }
    return $payload;
}
```

c. Langkah 2 - Mengganti requireAuth() menjadi requireAdmin() pada MahasiswaController

Pada store(), update(), dan destroy(), pemanggilan *Auth::requireAuth()* diganti *Auth::requireAdmin()*. Pemeriksaan dilakukan sebagai baris pertama, sebelum validasi data,

sehingga pengguna berrole user tidak dapat mengetahui apakah suatu id atau NIM ada dari pesan kesalahan validasi.

```
    public function store(): void {
        Auth::requireAdmin(); // sebelumnya Auth::requireAuth()
...

    public function update($id): void {
        Auth::requireAdmin();
...
    public function destroy($id): void {
        Auth::requireAdmin();
...
```

Karena akun hasil *POST /register* selalu berrole user, akun admin dibuat dengan mengubah kolom role lewat SQL

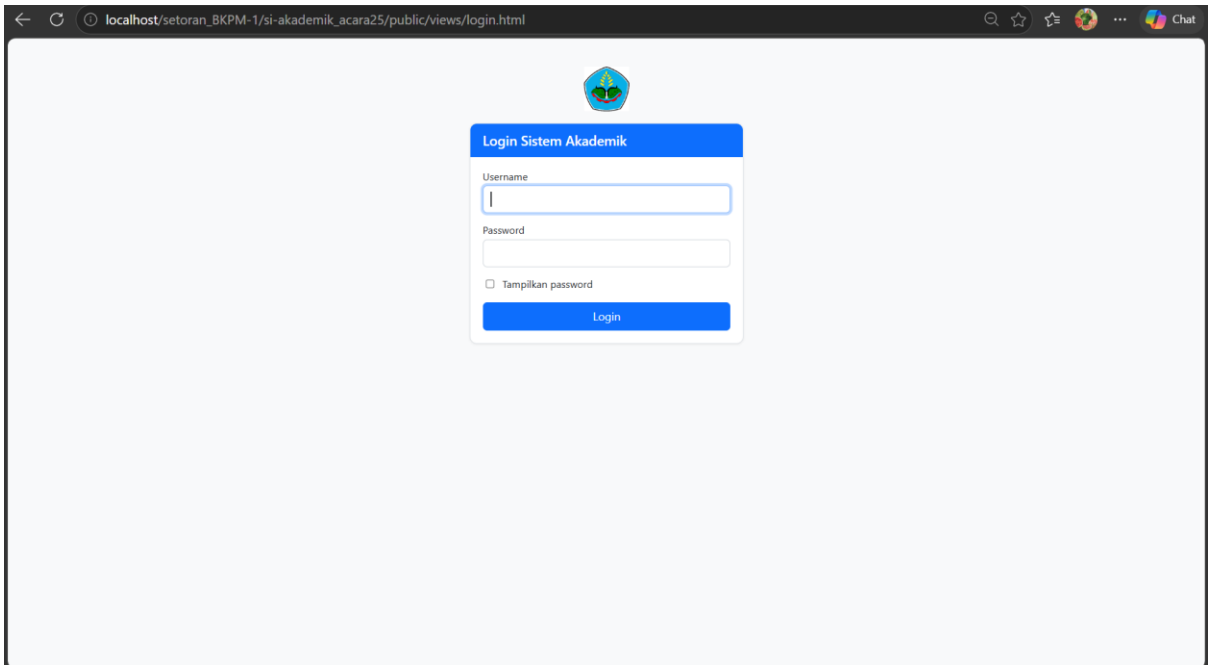
```
UPDATE users SET role = 'admin' WHERE username = 'budi';
SELECT id, username, role FROM users;
```

d. Langkah 3 - Membuat View login.html

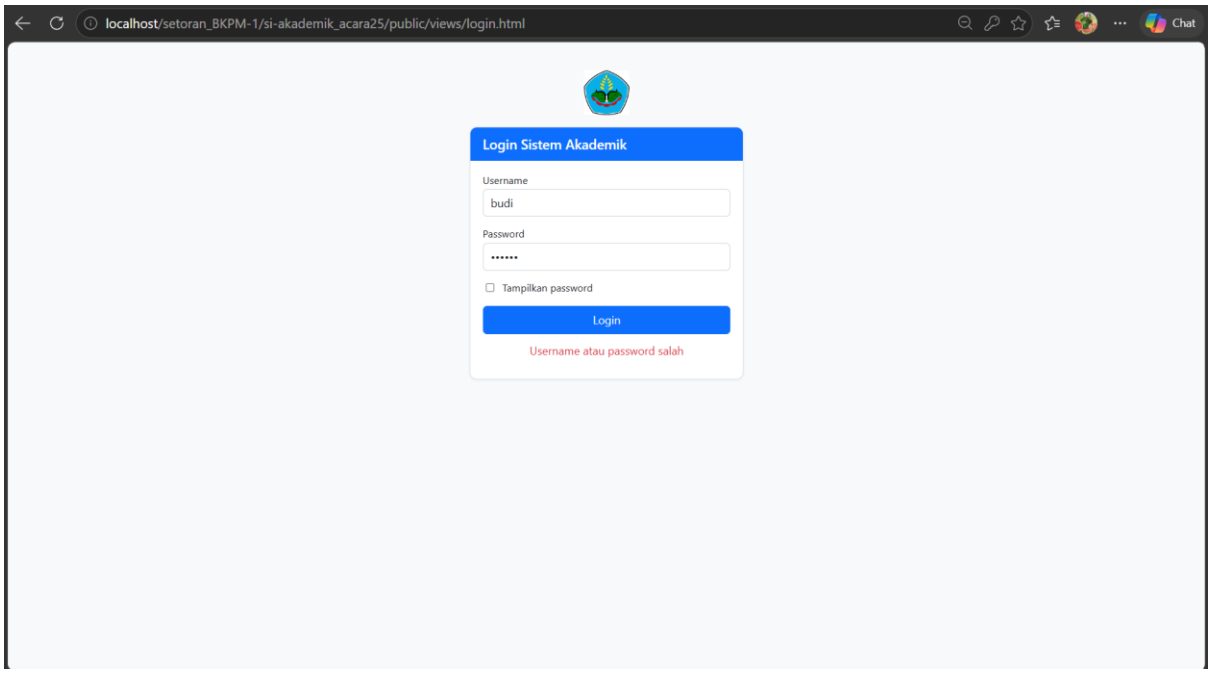
Alamat API dan pengelolaan sesi dipusatkan pada common.js agar login.js dan dashboard.js memakai satu sumber. Dibanding cuplikan modul, ada tiga penyesuaian: pesan galat ditampilkan pada elemen halaman (bukan *alert*), tombol dinonaktifkan selama permintaan berjalan, dan kegagalan jaringan ditangani dengan try-catch. Kredensial dikirim sebagai JSON ke POST /login; bila berhasil, token dan role disimpan lalu pengguna diarahkan ke dashboard.html.

```
form.addEventListener('submit', async (e) => {
    e.preventDefault();
    tombol.disabled = true;
    tampilkanPesan('Memproses...', 'info');
    try {
        const { ok, hasil } = await panggilApi('/login', {
            method: 'POST',
            body: { username: form.elements.username.value.trim(), password:
form.elements.password.value },
        });
        if (ok) {
            Sesi.simpan(hasil.data.token, hasil.data.role);
            window.location.href = 'dashboard.html';
            return;
        }
        tampilkanPesan(hasil.message || 'Login gagal', 'galat');
    } catch (error) {
        console.error('Gagal login:', error);
```

```
        tampilkanPesan('Tidak dapat menghubungi server. Pastikan server
berjalan.', 'galat');
    } finally {
        tombol.disabled = false;
    }
});
```



Gambar 1. Halaman login.html



Gambar 2. Login gagal (password salah) menampilkan pesan dari API

e. Langkah 4 - Melindungi View dashboard.html

Pemeriksaan token diletakkan pada elemen *script* di bagian *head* sehingga pengguna tanpa token dialihkan sebelum isi halaman ditampilkan. Di dashboard.js, token yang sudah lewat masa berlakunya (dibaca dari klaim *exp*) dibersihkan dan pengguna dikirim ke login.html?sesi=habis. Perlu dipahami bahwa pemeriksaan ini hanya mengatur tampilan: siapa pun dapat mengisi localStorage secara manual, sehingga perlindungan yang sebenarnya tetap berada di server (requireAuth dan requireAdmin).

```
<!-- dashboard.html, di dalam <head> -->
<script>
    if (!localStorage.getItem('token')) {
      window.location.href = 'login.html';
    }
  </script>
```

```
// dashboard.js
if (!Sesi.token()) {
    window.location.href = 'login.html';              // belum login
} else if (Sesi.kedaluwarsa()) {
    Sesi.hapus();
    window.location.href = 'login.html?sesi=habis';   // pernah login, masa
berlaku habis
}
```

Seluruh pemanggilan API yang memakai token melewati fungsi panggilApi(). Bila server menjawab 401 (misalnya token berumur lima menit sudah habis saat pengguna masih membuka dashboard), sesi dibersihkan dan pengguna kembali ke halaman login, bukan dibiarkan melihat pesan galat yang membingungkan.

```
// common.js (cuplikan)
async function panggilApi(path, { method = 'GET', body = null, auth = false }
= {}) {
    const headers = {};
    if (body !== null) headers['Content-Type'] = 'application/json';
    if (auth) headers['Authorization'] = `Bearer ${Sesi.token()}`;

    const response = await fetch(`${BASE_URL}${path}`, {
        method,
        headers,
        body: body !== null ? JSON.stringify(body) : undefined,
    });
    const hasil = await response.json();

    if (auth && response.status === 401) {
        Sesi.hapus();
        window.location.href = 'login.html?sesi=habis';
```

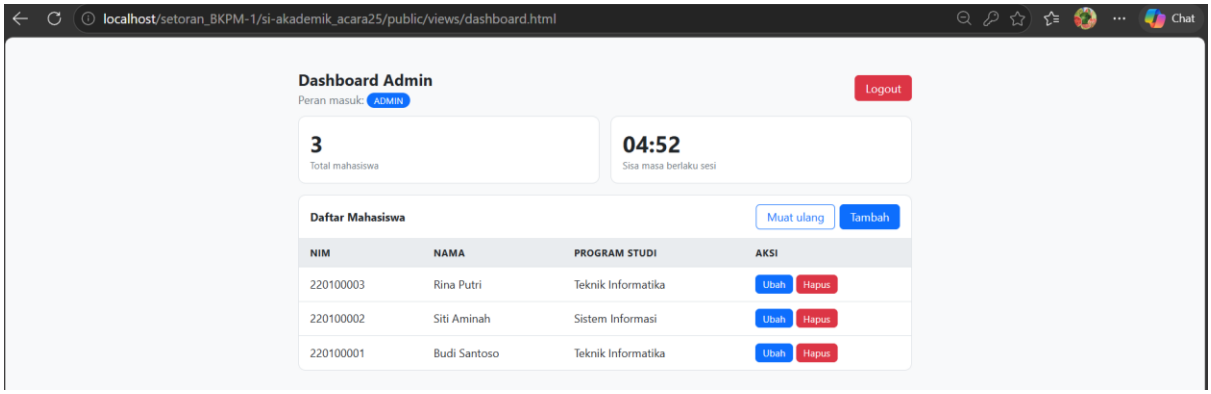
```
        throw new Error('Sesi berakhir');
    }
    return { status: response.status, ok: response.ok && hasil.success, hasil
};
}
```

f. Langkah 5 - Menampilkan Ringkasan Data pada Dashboard

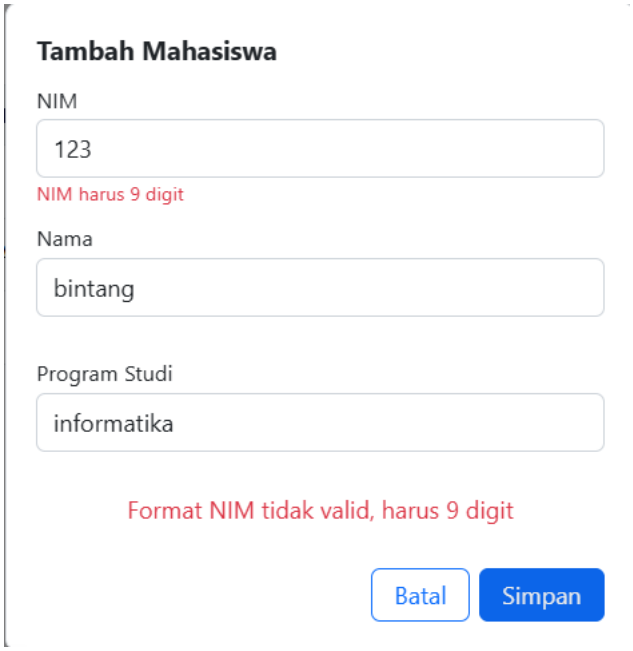
Dashboard memanggil GET /mahasiswa (MahasiswaController::index()) dan memakai panjang array *data* sebagai jumlah total mahasiswa. Daftar yang sama diisikan ke tabel, dan tabel dikosongkan tepat saat baris baru dipasang agar dua pemanggilan yang berjalan bersamaan tidak menggandakan baris. Selain jumlah mahasiswa, dashboard menampilkan peran pengguna dan hitung mundur sisa masa berlaku sesi.

```
 async function muatDataMahasiswa() {
    tampilkanStatus('Memuat data...', 'info');
    try {
        const { ok, status, hasil } = await panggilApi('/mahasiswa');
        if (!ok) throw new Error(hasil.message || `HTTP ${status}`);
        totalEl.textContent = hasil.data.length;
        if (hasil.data.length === 0) {
            tbody.replaceChildren();
            tampilkanStatus('Belum ada data', 'kosong');
            return;
        }
        // replaceChildren() saat render: dua pemanggilan bersamaan tidak
menggandakan baris
        tbody.replaceChildren(...hasil.data.map(buatBaris));
        tampilkanStatus('');
    } catch (error) {
        console.error('Gagal memuat data:', error);
        totalEl.textContent = '-';
        tampilkanStatus('Gagal memuat data. Pastikan server berjalan.',
'galat');
    }
}
```

Operasi Tambah dan Ubah memakai elemen *dialog* berisi form NIM, nama, dan program studi. Kesalahan validasi dari API (400, 422, 409) ditampilkan di bawah field yang bersangkutan sesuai isi *errors* pada respons. Hapus meminta konfirmasi terlebih dulu. Seluruh isi tabel dibuat dengan *textContent*, seperti pada Acara 24.



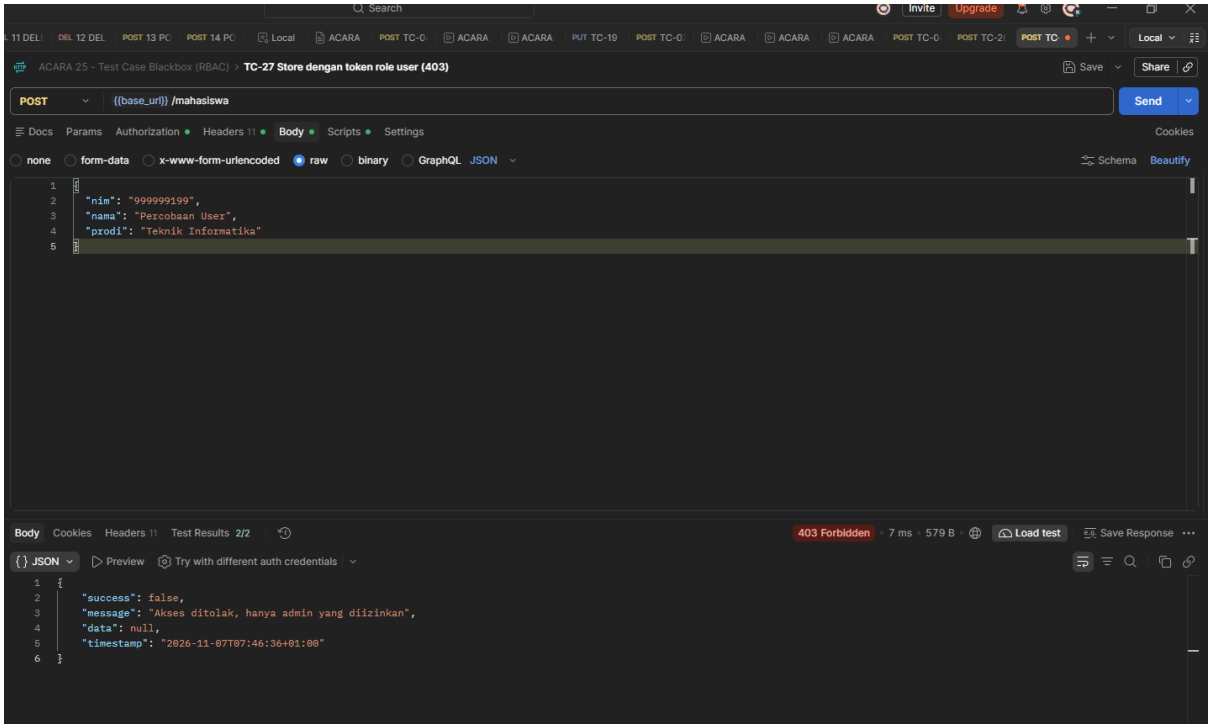
Gambar 3. Dashboard admin: ringkasan, tabel, dan tombol Tambah/Ubah/Hapus



Gambar 4. Form Tambah dengan pesan kesalahan validasi per field

g. Langkah 6 - Pengujian Alur Login dan RBAC

1. Login dengan akun role user. Tombol Tambah/Ubah/Hapus disembunyikan oleh tugas mandiri 2, sehingga permintaan tulis diuji langsung ke API dengan token user (Postman, TC-27 sampai TC-29). Hasilnya 403 dari Auth::requireAdmin() (Gambar 5).
2. Login dengan akun role admin dan pastikan seluruh fitur dashboard dapat diakses: tambah, ubah, hapus, dan muat ulang (Gambar 3).



Gambar 5. Postman: POST /mahasiswa dengan token role user menghasilkan 403

2. Tugas Praktik Mandiri

a. Tombol Logout

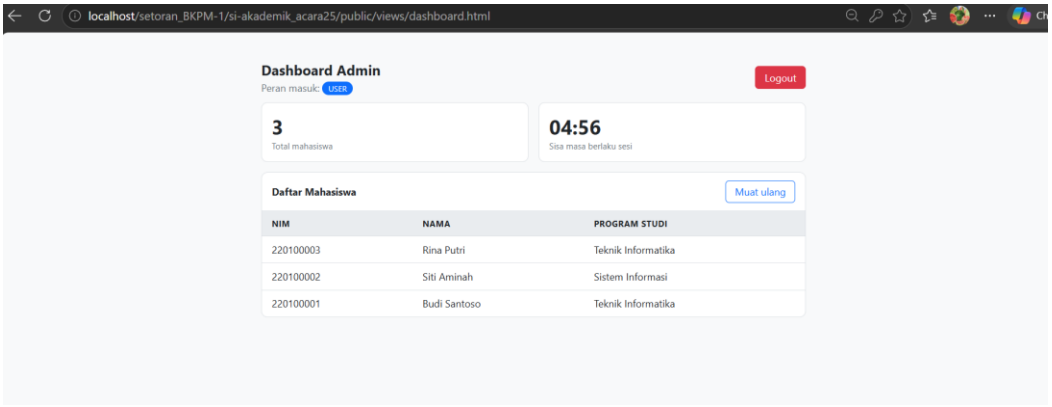
Tombol Logout memanggil fungsi yang menghapus token dan role dari localStorage lalu mengarahkan pengguna ke login.html. Karena JWT bersifat stateless, token yang sama masih sah di server sampai kedaluwarsa; Logout hanya menghapus token dari browser. Setelah Logout, membuka kembali dashboard.html langsung dialihkan ke halaman login.

```
function logout() {
    Sesi.hapus();
    window.location.href = 'login.html';
}
document.querySelector('#tombol-logout').addEventListener('click', logout);
```

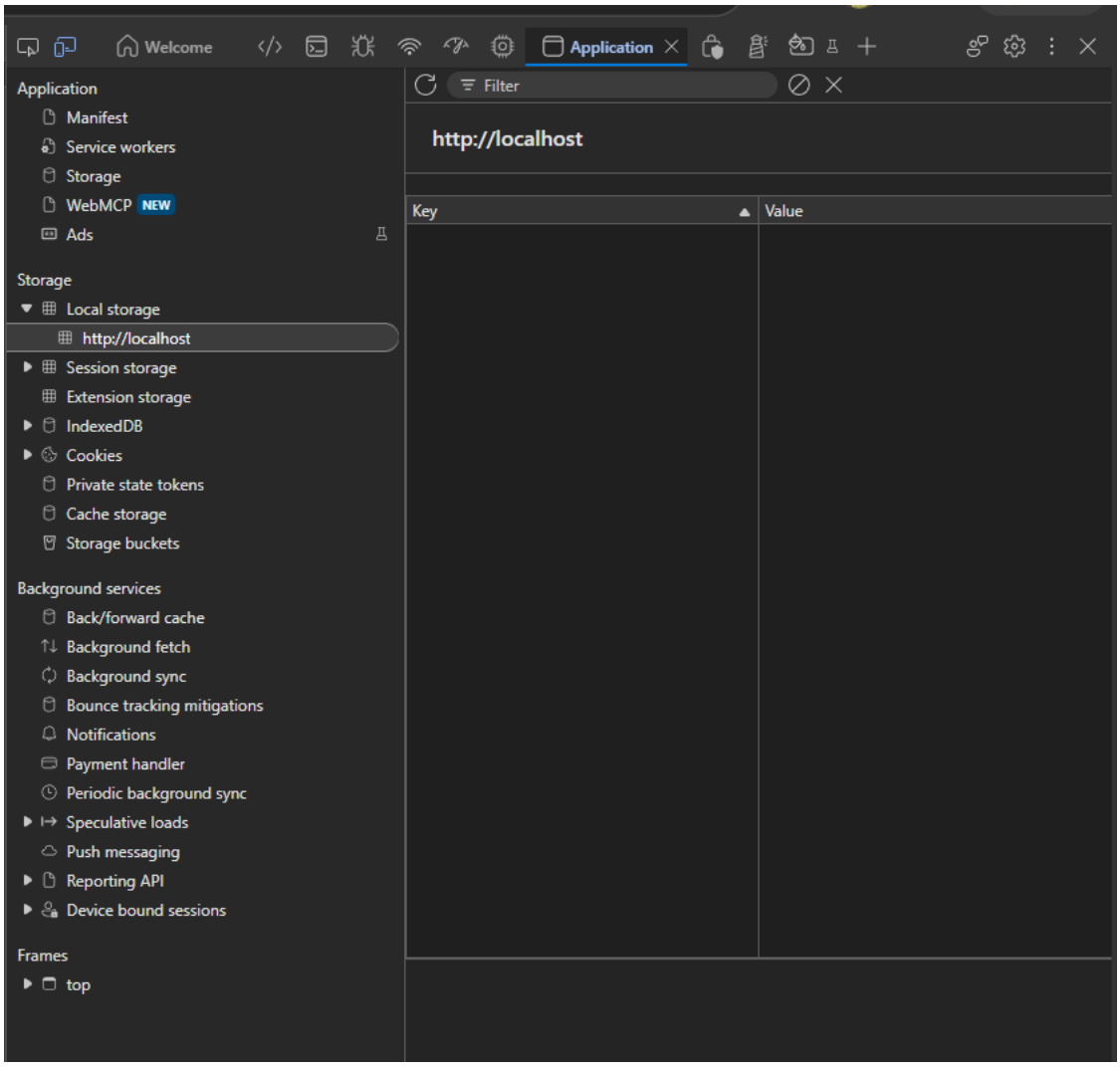
b. Menyembunyikan tombol bagi role user

Elemen yang hanya untuk admin diberi kelas *aksi-admin* dan disembunyikan bila role pada localStorage bukan admin. Tombol Ubah dan Hapus per baris tidak dibuat sama sekali untuk role user. Ini pelengkap tampilan; bila pengguna menyiasatinya, server tetap menolak dengan 403 dan dashboard menampilkan pesan penolakan.

```
const adalahAdmin = Sesi.role() === 'admin';
if (!adalahAdmin) {
    document.querySelectorAll('.aksi-admin').forEach((el) => { el.hidden =
true; });
}
```



Gambar 6. Dashboard role user: tombol Tambah, kolom Aksi, Ubah, dan Hapus tidak tampil



Gambar 7. Local Storage kosong setelah Logout (token dan role terhapus)

3. Pengujian

Pengujian dilakukan pada tiga lapisan. API diuji dengan Collection Postman yang dijalankan memakai Newman terhadap server PHP dan MariaDB nyata: 25 test case Acara 23 sebagai

regresi (login memakai akun admin) dan 4 test case baru untuk RBAC. Logika halaman diuji dengan lingkungan DOM jsdom yang memanggil server yang sama, mencakup 27 skenario. Tampilan akhir dan Developer Tools tetap diperiksa pada browser (Gambar 1 sampai 7). Seluruh pemeriksaan sesuai harapan (Tabel 3).

Tabel 3. Hasil pengujian

No	Skenario	Hasil yang diharapkan	Hasil
1	Login password salah	Pesan "Username atau password salah", tidak ada token tersimpan	Sesuai
2	Login akun admin	Token dan role tersimpan, menuju dashboard.html	Sesuai
3	Server tidak terjangkau saat login	Pesan gagal terhubung, tombol aktif kembali	Sesuai
4	Membuka halaman login saat sudah login	Langsung menuju dashboard.html	Sesuai
5	Membuka dashboard tanpa token	Diarahkan ke login.html	Sesuai
6	Token kedaluwarsa	Sesi dibersihkan, menuju login.html?sesi=habis	Sesuai
7	Dashboard admin dengan 3 data	Total 3, tiga baris tabel, tombol Tambah/Ubah/Hapus tampil	Sesuai
8	Tambah data valid	Dialog menutup, total 4, baris baru tampil di atas	Sesuai
9	Tambah NIM duplikat / 3 digit / kosong	Pesan 409, 422, atau 400 tampil pada field terkait	Sesuai
10	Ubah dan hapus data	Tabel dan total diperbarui	Sesuai
11	Tombol Muat ulang ditekan	Data dimuat ulang tanpa menggandakan baris	Sesuai
12	Tombol Logout	Token dan role terhapus, menuju login.html	Sesuai
13	Dashboard role user	Total dan tabel tampil; Tambah, Aksi, Ubah, Hapus tersembunyi	Sesuai
14	Role user memaksa POST (tombol disiasati)	Server 403, pesan penolakan tampil, data tidak bertambah	Sesuai
15	Nama berisi tag HTML (<img onerror=...>)	Tampil sebagai teks, tidak dieksekusi	Sesuai
16	Token bertanda tangan palsu pada operasi tulis	401, sesi dibersihkan, menuju login.html?sesi=habis	Sesuai
17	Tabel kosong; server mati saat memuat	"Belum ada data", total 0; pesan "Gagal memuat data"	Sesuai
18	POST/PUT/DELETE /mahasiswa dengan token user (TC-27 sampai TC-29)	403 - Akses ditolak, hanya admin yang diizinkan	Sesuai
19	POST/PUT/DELETE tanpa token atau token rusak	401	Sesuai
20	Regresi 25 test case Acara 23 (akun admin)	25 Pass, 0 Fail	Sesuai

Ringkasan Newman: 31 request dan 35 assertion dijalankan tanpa kegagalan. Ringkasan jsdom: 27 skenario, 27 Pass.

4. Pembahasan

Pemisahan 401 dan 403 membuat penyebab penolakan jelas bagi klien: 401 berarti pengguna perlu login ulang, 403 berarti login ulang tidak akan membantu. Karena requireAdmin() dijalankan sebelum validasi data, TC-28 dan TC-29 tetap menghasilkan 403 walau id yang dimaksud tidak ada; dengan begitu respons tidak membocorkan keberadaan data kepada pengguna yang tidak berhak.

Perubahan ini berdampak pada pengujian sebelumnya. Collection Acara 23 melakukan login memakai akun budi untuk operasi tulis; setelah requireAdmin() dipasang, akun itu harus berrole admin, jika tidak test case POST, PUT, dan DELETE akan memperoleh 403. Karena itu ditambahkan db_seed_acara25.sql, dan akun role user untuk TC-26 sampai TC-29 diambil dari akun acak yang dibuat TC-07 sehingga tidak perlu data tambahan.

Pengujian jsdom menemukan dua kekurangan pada kode sebelum diperbaiki. Pertama, hitung mundur sesi mengalihkan pengguna yang belum login ke login.html?sesi=habis sehingga muncul pesan "Sesi berakhir" padahal pengguna belum pernah login; hitung mundur kini hanya berjalan bila token ada. Kedua, dua pemanggilan muatDataMahasiswa() yang berjalan bersamaan dapat menggandakan baris karena tabel hanya dikosongkan di awal; tabel kini dikosongkan saat baris dipasang.

Soal keamanan sisi klien: menyembunyikan tombol dan memeriksa localStorage hanya mengatur tampilan, karena pengguna dapat mengubah localStorage lewat Developer Tools. Pembatasan yang menentukan adalah requireAdmin() di server, dan inilah yang diuji pada TC-27 sampai TC-29. Token pada localStorage dapat dicuri bila ada celah XSS; penggunaan textContent (pengujian nomor 15) menutup celah itu pada isi tabel, tetapi cookie HttpOnly lebih tahan terhadap pencurian token bila diterapkan pada pengembangan berikutnya.

Keterbatasan: Logout tidak mencabut token di server sehingga token yang sama tetap berlaku hingga kedaluwarsa lima menit; endpoint /buku masih publik dan belum memakai requireAdmin(); akun admin hanya dapat dibuat lewat SQL karena register selalu menghasilkan role user; BASE_URL masih ditulis tetap pada common.js dan app.js; header CORS masih bernilai * (sebaiknya dibatasi pada produksi); dan pengujian jsdom tidak menggantikan browser sungguhan, karena itu tampilan diperiksa langsung pada Gambar 1 sampai 7.

KESIMPULAN

1. Method Auth::requireAdmin() berhasil ditambahkan dan dipakai pada store(), update(), dan destroy(); token role user dijawab 403, token tidak sah dijawab 401, dan pembacaan data tetap publik.
2. Halaman login.html menyimpan token dan role ke localStorage, dan dashboard.html hanya dapat dibuka dengan sesi yang masih berlaku; sesi yang habis mengembalikan pengguna ke halaman login.
3. Dashboard menampilkan jumlah total mahasiswa serta tabel dengan operasi Tambah, Ubah, dan Hapus bagi admin; pesan kesalahan validasi API tampil pada field terkait.
4. Pengujian menunjukkan RBAC bekerja (4 test case baru Pass) dan 25 test case Acara 23 tetap Pass dengan akun admin, serta 27 skenario halaman Pass.
5. Tugas mandiri selesai: tombol Logout dan penyembunyian tombol Tambah/Ubah/Hapus bagi role user, dengan proteksi utama tetap di sisi server.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). *Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805)*. Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.